

Speakgen — plans and implementation

Pricing tiers, the staged build, and the storage cost analysis. Agreed 17 September 2026.

Pricing tiers

The segmentation axis is **individual teacher versus academy**, not feature counts. That is where willingness to pay actually jumps: a freelance teacher pays personally and has a handful of classes, while an academy has a budget line, several teachers, and cares about brand consistency.

Tier	Monthly	Yearly	What it includes
Visitor no account	—	—	House exams and questions, fully runnable. Nothing saved.
Teacher Free	€0	—	Unlimited questions, 1 exam, 3 practices, share links. Ads shown.
Teacher Pro	€5	€49	Unlimited exams and practices, PDF export, fork house exams, no ads.
Academy	€29	€290	Pro for 5 teachers, shared question bank, school branding on shared links. Extra seats €5/month each.
AI credits add-on, later	—	—	About €5 per 100 generations. Available on any paid plan.

Yearly is roughly two months free. Teaching is termly, so an annual option matters more than it would elsewhere: monthly plans get cancelled every July and many of those people never return.

What is deliberately not charged for

- **Questions stay unlimited on every tier.** A teacher's question bank is the switching cost that keeps them. Capping it attacks the mechanism that drives retention.
- **Share links stay free.** They are the distribution channel — every link a free teacher sends lands in front of a class, and some of those are other teachers.
- **House content stays open to visitors.** It is the shop window, and it is the only thing a prospect sees before deciding.

Decisions

Advertising

Approved, but confined to **signed-out marketing pages and the free teacher's own dashboard**. Never on the exam runner, never on a shared student link.

- A shared link is both the distribution channel and the surface the Academy tier sells (school branding). Ads there poison both — and you cannot sell that space twice.
- Students are often 15–17 and sometimes younger, which drags in ad-targeting rules for minors and gives school IT a reason to block the site.
- The exam runner is a timed speaking activity. An ad loading mid-turn is disruptive.
- Revenue is small: roughly €1–3 per thousand pageviews. One academy subscription beats the entire ad line. Treat it as a conversion nudge, not income.

Build it behind a route allowlist and a kill-switch environment variable so a new page is ad-free by default and the whole thing can be withdrawn.

AI

Deferred, but the schema plumbing goes in early. The framing is **assisted authoring, not generation**: Cambridge formats are rigid, and a plausible-but-wrong Part 3 damages trust in exactly the way a teacher cannot afford in front of a class. Output is constrained by the existing blueprint and lands as a draft with the `needs_review` flag already in the schema. Priced as credits rather than a tier, because it carries real marginal cost.

VAT — still open

Euro pricing to EU customers means VAT at the customer's local rate. Two routes: **Stripe** plus Stripe Tax (~0.5%), where you remain the merchant and must register for VAT OSS and file returns yourself; or a **merchant of record** such as Paddle or Lemon Squeezy (~3% more), which handles registration, collection and filing entirely. Recommendation is merchant of record — the extra margin is the cheapest accountant available. **Settle this before stage 4**, because it shapes that stage.

Implementation stages

In dependency order. Stages 1 and 2 are small and additive; stage 3 is the first change a user sees.

1 Profiles for everyone

Upsert a `user_profiles` row when a session is first resolved, and backfill anyone created since migration 006. No behaviour change, but every stage below reads this table and a missing row is not the same as a free plan.

Blocks everything else

2 Plan model in the schema

Widen plan to free / pro / academy. Add a subscriptions table for provider ids, status, interval, period end and cancellation state, kept separate from the profile because it is external state with its own lifecycle. Add the AI credit balance and an events ledger now: retrofitting a ledger onto a bare counter later is the painful version. Entitlements live as a TypeScript map keyed on plan, not a permissions table.

Additive, nothing enforced

3 Enforce the free limits

A single `assertWithinPlan()` call at the create routes, with upgrade prompts where a limit bites. This ships before payments exist: everyone is on free, the limits are real, and Upgrade points at a waitlist — so the numbers get tested before any billing is built.

First visible change

4 Payments

Provider decision first, then monthly and yearly prices, hosted checkout, a customer portal for cancellations, and a webhook that is the only thing that ever changes a plan — never the client, never the checkout redirect. Idempotent, because webhooks retry.

The long pole

5 Advertising

One `AdSlot` component returning nothing for paid users, on excluded routes, and when the kill switch is off. Route allowlist rather than blocklist. Plus a consent banner for the EEA, which is needed regardless.

Can run in parallel with 3 and 4

6 Academy tier

Adopt the organization, member and invitation tables that Neon Auth already ships and that are sitting unused in the database, rather than inventing new ones. Content gains an optional organisation owner alongside its user owner; seats drive billing quantity.

Largest chunk

7 School branding

Logo and colours applied to shared student pages. Logo upload to Vercel Blob — see the cost rules overleaf.

Blocked on share links

8 AI

Generation constrained by the blueprint rather than free-form, output flagged for review, credits spent through the ledger built in stage 2.

Last

A parallel track feeds this: **practices** → **share links** → **branding**. Only stage 7 is blocked on it.

Vercel Blob — what it actually costs

Relevant to stage 7, where school logos are stored. Checked against the Vercel documentation on 17 September 2026.

Blob bills on **six** dimensions, and two of them are not labelled Blob at all — they appear on the CDN line.

Dimension	Charged for
Storage	Monthly average store size, in GB-months
Simple operations	Cache miss on a URL, or a head() call
Advanced operations	put(), copy() and list() — the expensive ones
Blob data transfer	Downloads and views
Edge requests	Every URL access, hit or miss. Billed at standard CDN rates.
Fast origin transfer	Cache misses only. Also billed at standard CDN rates.

The traps

- **list() is an advanced operation at \$5.00 per million**, twelve times the price of a simple one. Calling it in a request path to find a file is the expensive mistake.
- **Browsing your own store in the Vercel dashboard costs money.** Each interaction counts as an advanced operation.
- **Server uploads cost more than client uploads.** A file that transits your own function incurs fast data transfer; a direct client upload does not.
- **Private blobs are dearer to serve.** They go through a function, so you pay the store-to-function fetch and the function-to-browser response. Public blobs go browser-direct, and blob data transfer is about three times cheaper.
- **Multipart uploads bill per part**, plus one to start and one to complete.
- **On the Hobby plan it is a hard stop, not an overage.** Exceed the limits and Blob access disappears for thirty days — disqualifying for anything customers pay for.
- Deletions are free.

For this use case it rounds to zero

School logos are the best-case shape: tiny, static and cacheable to near a hundred per cent. A hundred academies with a 50 KB logo each is **5 MB stored**, or about a hundredth of a cent per month. Even at half a million shared-link views, a cached logo generates almost no simple operations, and the resulting transfer sits inside the allowance.

Four rules keep it there

- **Public store, not private** — a school logo is not a secret, and this keeps a function out of the delivery path.
- **Client uploads**, so the file never transits an API route.
- **Cap and resize on upload** (around 200 KB, 512 px), so a school cannot upload a six-megabyte PNG and move your transfer costs.
- **Store the URL on the organisation row** and never call list() at request time.

Source: vercel.com/docs/vercel-blob/usage-and-pricing